Email capabilities fact-check: E01–E07 with a Gmail plugin

Overall assessment

This assessment assumes that a user connects their Gmail account through a Situate plugin. With working search and full-content reading exposed to the conversation, existing model tools can support understanding, drafting, finding information, follow-up analysis and qualitative pattern recognition without seven dedicated product features. Saving account drafts, changing the mailbox and sending messages require additional provider operations; account sign-in alone does not supply them. Both the developing main application and the separate macOS prototype have plugin execution foundations, but no concrete Gmail plugin was inspected. The judgments below are therefore conditional technical feasibility, not verified Gmail support or a guarantee of answer quality.

Capabilities examined

These seven capabilities define the assessment subjects, not implementation evidence. Their scope is independent of the Gmail-plugin assumption and the findings below.

E01 — Understand messages and threads

Understand, summarize and answer questions about individual messages and longer correspondence: what the exchange concerns, what was agreed and what remains open. Include technically accessible attachments. Distinguish full message content from previews and complete threads from isolated messages, and identify relevant attachment limits.

E02 — Compose email

Draft and revise new messages and replies from bullet points or the previous exchange, taking tone, length and personal writing style into account. Save and update drafts in the email account, distinguishing real account drafts from text produced only in chat. Style preferences may be supplied in a Markdown file; assess whether the assistant can access and use them without assuming a dedicated style feature.

E03 — Find information in email

Find messages and specific information by meaning and context, even without a known subject or exact wording. Also narrow the selection by age/date, sender, subject or attachments. Assess the search actually exposed and whether enough content can subsequently be read for a meaningful selection; no particular search technology is required.

E04 — Organize the mailbox

Classify and group messages by content and requested criteria. With suitable tools, create, assign or remove labels or provider-specific filing attributes; change read status and markings; archive or move messages to trash, including multiple matching messages. Formulate sorting rules and apply them once to existing mail. Assess materially different sub-actions separately, distinguish suggestions from actual account changes, and examine persistent rules for future incoming mail separately from one-off batches.

E05 — Send and forward email

Actually send prepared new messages and replies, and forward existing messages. Follow the path from the model tool to the email provider, including recipients, reply linkage and the reported outcome. Draft creation or a simulated sending confirmation does not establish sending; identify relevant forwarding and attachment limits.

E06 — Follow up open correspondence

Across multiple threads, identify who is still waiting for an answer, decision or promised follow-up: who is waiting for the user, and whom the user is waiting for. Assess access to sent and received messages, thread association, freshness and traceable source references. Read status or the presence of any reply does not establish resolution. An on-request assessment is sufficient; autonomous reminders require additional capabilities.

E07 — Recognize communication patterns

Across multiple correspondences, examine relationships between the wording of the user's emails and observable reactions, such as whether a specific question is followed by a reply and how soon it arrives. Assess message selection, reply association and analysis of language features. Distinguish qualitative hints from a few examples from reproducible response rates or time comparisons for a defined collection. General model and calculation tools may suffice without a dedicated analytics feature. Observed associations do not establish causation; the capability is context-neutral.

Assumption, variants and execution path

Assumed: a compatible plugin is installed, enabled and authorized for the selected Gmail account. **Not automatically assumed:** search/read access, complete threads, attachment handling, write permissions, sending or permanent filters. Each row identifies the operations needed; their presence in the actual plugin remains **UNKNOWN** until inspected.

The intended path is: user request → model selects a registered plugin tool → plugin calls Gmail using the authorized account → provider data or outcome returns to the conversation. Situate's existing model, file and calculation tools can then process that result. An account connection or a success message without the corresponding provider operation is not evidence that this path works.

Product variant	Inspected foundation and relevant boundary
Main application: macOS/iOS	Captures enabled workspace-plugin tools alongside general file tools and executes plugin JavaScript. macOS also offers shell scripts; iOS does not, so deterministic calculations there require callable plugin code. Generic credential storage and HTTP exist, but neither supplies Gmail authorization or mailbox semantics by itself.
Separate prototype: macOS	Loads active plugins from private application storage into the conversation and offers file/search/command tools plus generic account authorization and token renewal. Its bundled integrations cover Apple Calendar, Reminders and Notes, not Gmail; additional installed plugins were not inspected.

The Gmail assessment applies to either variant **only if its compatible plugin exposes the required operations**. A prototype plugin is not assumed to run unchanged in the main application, nor does a missing main-app feature disprove prototype support. Other email providers are outside this assumption.

E01–E07 under the Gmail assumption

YES, conditional means that the complete stated path is technically feasible when the listed plugin operations exist; it does not assert that an uninspected plugin provides them. **PARTIAL** separates available model processing from unknown account operations. **UNKNOWN** means the connection assumption does not establish the necessary operation. Evidence references describe the host foundations, not Gmail-specific implementation.

E01 — Understand messages and threads

YES, conditional

The model can summarize, explain agreements and identify open questions if the plugin supplies full message bodies and all relevant thread members, rather than previews or one isolated message. Attachments also need retrieval and a readable format; unsupported or protected content remains excluded.

Conditional inference from code: M1, M3; P1–P3

E02 — Compose email

PARTIAL; account drafts UNKNOWN

New messages, replies and revisions can use retrieved correspondence and a Markdown style file explicitly read from the local workspace. Saving and updating a real Gmail draft becomes **YES** only with corresponding plugin operations; chat text and local draft files are not account drafts.

Conditional inference from code: M1–M3; P1–P2

E03 — Find information

YES, conditional

Search, result-page traversal and full-content reading let the model vary queries and choose relevant messages by meaning; a dedicated semantic index is unnecessary. Date/age, sender, subject and attachment criteria require suitable search options or enough retrieved metadata/content to filter afterward; a capped first page cannot establish complete coverage.

Conditional inference from code: M1–M3; P1–P2

E04 – Organize the mailbox

PARTIAL; account changes UNKNOWN

The model can classify retrieved messages and formulate sorting rules, but label creation/assignment/removal, read/star changes, archive/trash and one-off multi-message actions require corresponding write operations and permissions. Permanent rules for future mail require a separate implemented operation; neither a successful batch nor Gmail sign-in proves it.

Conditional inference from code: M1–M2; P1–P2

E05 – Send and forward

UNKNOWN; YES with a verified send path

Sending new mail and replies requires a registered tool that passes recipients, content and reply linkage to Gmail and returns the actual provider outcome, not a simulated confirmation. Forwarding and outgoing attachments must also be implemented; draft creation alone establishes neither.

Conditional inference from code: M1–M2; P1–P2

E06 – Follow up open correspondence

YES, conditional

On request, the model can assess who owes whom an answer or decision if it can retrieve current incoming and sent messages, associate their threads and cite message identifiers or links. It must evaluate the content, not equate unread status or any reply with resolution; autonomous reminders are an additional capability.

Conditional inference from code: M1–M3; P1–P2

E07 – Recognize communication patterns

PARTIAL; quantitative results conditional

Qualitative comparisons of wording and reactions become **YES** with readable, related correspondence. Reproducible response rates and time comparisons also become **YES** with complete selection, timestamps, explicit reply-matching rules and callable calculation tools; few examples do not establish a mailbox-wide pattern or causation.

Conditional inference from code: M1, M4; P1–P2

Three important distinctions

1. **Connection versus capability.** Inspect the tools actually registered in the conversation and their granted permissions, not just the account's "connected" status. A missing operation may need only configuration if already implemented, otherwise plugin development; no conclusion about Gmail's own limits follows from an absent Situate tool. This also applies to persistent filters.
2. **Full, current data versus excerpts or copies.** Provider pagination and full-thread retrieval belong to the plugin. Pasted, exported or cached mail is only a snapshot unless refreshed; search previews cannot substitute for message bodies. Main-app local file reads are paginated at up to 32 KiB, with a 16 MiB input ceiling and a 2 MiB image ceiling; its generic plugin HTTP bodies are bounded to 1 MiB UTF-8 text. These are different limits, not a verified Gmail message limit. Encoded email bodies and PDF/Office attachments may need extraction; images require a suitable model. Prototype reads are bounded too, and installed converters cannot be assumed.
3. **Feasible processing versus validated conclusions.** General model and script tools can suffice for E06/E07 without special features. Best-effort examples are not a reproducible measurement: document the selected population, observation cutoff, missing records and reply matching. The analysis must separate unresolved commitments from merely unanswered messages, and observed correlations from causes.

Evidence and verification

Revisions and local differences

- **Main application:** branch `use-case-validation`, commit `27b6831c3e55b2de4a59d6cd32cf1935d373338d`. No tracked product changes. Untracked documentation comprises this report and its PDF, plus the unrelated note/task-management fact-checks in Markdown and PDF; those unrelated files are outside this review.
- **Prototype:** separate "The Maybe Company" repository, branch `main`, commit `53863aa6be03a1033d8da65801f84399799a5a0c`; source root `second-brain/Prototype/`. At source inspection, pre-existing differences were modified `Embedded/PiBridge`, staged/untracked `.idea/` files and untracked `PRODUKTUEBERGABE.md`; inspected product source files were unchanged. The modified embedded executable was not qualified.
- Prototype `PiBridge/package.json:8-11` requests Pi **0.85.1**, while the installed coding-agent package reported **0.84.3**. Prototype test results below apply to that local dependency environment, not a fresh pinned build.

Shared code references

`M` paths are relative to this repository; `P` paths are relative to the separate prototype source root. These establish the reusable foundations; **none is a verified Gmail implementation**.

- **M1 – Conversation/tool connection:** `client/app/src-tauri/src/ai_conversations.rs:269-300,573-607,866-909`; `client/app/src-tauri/src/ai_builtin_tools.rs:9-130`. Chat submission: `client/app/src/ai/chat.ts:874-923`, `client/app/src/ai/native-conversation.ts:284-291`.
- **M2 – Plugin/provider boundary:** `client/crates/situate-core/src/plugins.rs:220-259`; `client/app/src-tauri/src/ai_executor.rs:1157-1208`; `client/app/src-tauri/src/plugin_host.rs:1103-1230`. Workspace plugin inventory, captured tool invocation and generic credentials/HTTP.
- **M3 – Local reading and writing:** `client/app/src-tauri/src/file_tool.rs:215-340`; `client/app/src-tauri/src/files.rs:1438-1567`; `client/crates/situate-core/src/content/agent_tools.rs:41-75`. Literal search, continuation, bounded text/images and local versioned writes, including readable style files.
- **M4 – Calculation routes:** `client/app/src-tauri/src/shell_host.rs:32-57`; `client/app/src-tauri/src/files.rs:1366-1388`; `client/app/src/plugins/executor-worker.ts:69-91,132-150`. macOS shell execution, its absence on iOS and callable plugin JavaScript.
- **P1 – Conversation/general tools:** `PiBridge/situate-tools.ts:1-2`; `PiBridge/bridge.ts:1899-1928,2225-2229,3605-3637`. Tool defaults, session registration and request submission.
- **P2 – Plugin loading/context:** `PiBridge/bridge.ts:1350-1370,1668-1685`; `PiBridge/plugin-resource-loader.ts:22-35`; `PiBridge/bundled-apple-plugins.ts:19-46`. Working instructions, active plugin loading and the bundled Apple set.
- **P3 – Images/account infrastructure:** `PiBridge/image-content.ts:16-25,55-74`; `PiBridge/situate-host-api.ts:257-287`; `Situate/PluginConnectorHost.swift:214-240`. Image handling and generic account operations, not email endpoints.

Documentation and executed checks

Approved requirements are not delivery evidence: `requirements/data/requirements/REQ-0002-search-and-read-email.md:12-16,26-37` and `requirements/data/requirements/REQ-0003-save-email-drafts.md:12-16,26-38`. Prototype `PiBridge/SituatePluginDocumentation/v4/connectors.md:74-76` describes Gmail setup; the separate repository's `second-brain/demo-data/imagine/setup/gmail.md:1-5,77-103` describes a synthetic-mail importer, not a Situate conversation tool.

Previously executed prototype checks: from `PiBridge/`, `bun test plugin-resource-loader.test.ts plugin-oauth.test.ts conversations-bridge.test.ts`: **8 passed, 0 failed**. They check temporary plugin loading, OAuth manifest validation and a real bridge process with a **simulated local model**. They were not rerun for the assumption-only revision and do not test Gmail or email-analysis quality.

Repository checks for the revised report: `node tools/test.mjs --quick --changed` passed **2/2** suites; `node tools/test.mjs --feedback --changed` passed **17/17**, including formatting and document/brand contracts. The earlier feedback run had failed formatting in this report and the unrelated task-management report; that failure did not recur. Unrelated untracked PDFs broadened the changed-file selection beyond documentation checks. These general checks do not qualify Gmail support.

No concrete Gmail plugin, live mailbox, real model, GUI or native iOS journey was exercised. The conditional assessments are not new practical test results; actual plugin operations, granted permissions and end-to-end quality remain open.

Next steps

1. **Setup/access:** inspect the installed Gmail plugin's code, registered tools and non-secret configuration for the selected product variant and authorized account; do not supply tokens or personal mail.
2. **Further verification:** use a synthetic account to check full messages/threads, attachments, pagination, draft updates, mailbox changes and provider-confirmed sending, then assess E06/E07 against known outcomes.
3. **Product change:** add only operations missing from the chosen plugin; treat permanent filters and autonomous reminders separately from one-off sorting and on-request analysis.